



## 1 ROS2-Domain IDs / Local

Damit sich verschiedene Instanzen von ROS im selben Netzwerk nicht gegenseitig finden, muss in jedem Terminal beim öffnen einer der folgenden Befehle ausgeführt werden:

```
export ROS_LOCALHOST_ONLY=1
export ROS_DOMAIN_ID=<unique number between 0 and 101>
```

Falls sie sich für eine *ROS\_DOMAIN\_ID* entscheiden sprechen sie diese mit den anderen Teams ab.

Wenn sie den Befehl nicht mit jedem Terminal erneut eingeben möchten können sie diesen in das *bashrc* -file aufnehmen. Genaueres dazu finden sie hier.

## 2 Vertraut machen mit der Installation

Auf ihrem Praktikumsrechner ist ROS2-Humble installiert. Um ROS direkt zu sourcen können sie also den folgenden Befehl verwenden:

```
source /opt/ros/humble/setup.bash
```

Die Middleware Dynamic Swarms Crazyflies (im Ordner */ds-crazyflies*) wurde dabei bereits auf diese Ros-Installation aufgebaut. Wollen sie diese also direkt verwenden genügt folgender Befehl (aus dem home-Verzeichniss):

```
source ds-crazyflies/install/setup.bash
```

Die Beispielanwendung *ds.tha.example* wurde anschließend auf das Dynamic Swarms Package gebaut. Hier genügt also der Befehl:

```
source ds_tha_example/install/setup.bash
```

## 3 Verwendung der Simulation

Starten sie zunächst die Webots Simulation (Windows-Taste ... *webots* eintippen). Dabei sollte automatisch die passende Welt geladen werden. Ist dies nicht der Fall finden sie diese im Home-Verzeichniss unter *crazywebotsworld* (Es ist die selbe wie sie bereits in der Übung verwendet wurde). Im Webots Terminal sollte die Meldung *controller disconnected* auftauchen.

Sourcen sie anschließend in einem Terminal das *ds.tha.example* und führen sie folgenden Befehl aus:

```
ros2 launch tha_example tha_framework.launch.py type:=2 backend:=webots id:=0
```

Für Informationen über die Argumente können sie folgenden Befehl ausführen:

```
ros2 launch tha_example tha_framework.launch.py --show-args
```

Nun sollte in Webots die Meldung erscheinen *controller connected*. Sie können jetzt in einem weiteren Terminal die folgenden Befehle benutzen:

```
ros2 topic pub /safeflie0/takeoff std_msgs/Empty
ros2 topic pub /safeflie0/send_target ....
ros2 topic pub /safeflie0/land std_msgs/Empty
```

Starten sie damit die Simulierte Drohne, setzen sie anschließend ein neues Ziel. Landen sie. Falls sie die Drohne wieder auf dem Nullpunkt haben wollen können sie die Webots-Simulation zurücksetzen. Sie müssen dabei auch das Framework einmal schließen und dann wieder erneut verbinden.

Sie können sich nun auch das Beispiel in *ds\_tha\_example/tha\_example/tha\_example/tha\_flie.py* anschauen und verstehen. Dafür ist auf ihrem Praktikumsrechner bereits VSCode vorinstalliert.

## 4 Crazyflie Client

Führen sie in einem neuen Terminal folgenden Befehl aus (aus dem home-Verzeichnis):

```
python3 crazyflie-clients-python/bin/cfclient
```

Mit diesem Client können sie sich mit einer Crazyflie verbinden, Parameter setzen, LogDaten auslesen, sowie Tests durchführen. Machen sie sich mit der UI vertraut. Speziell können sie unter dem Reiter *View* verschiedenen Tabs und Toolboxes öffnen, mit denen verschiedenen Funktionalitäten der Crazyflie verwendet werden können.

Eine hilfreiche Übersicht über die Funktionalitäten finden sie auch nochmal hier.

## 5 Logging

Verbinden sie eine echte oder Simulierte Crazyflie mit dem Framework.

Benutzen sie das von der Crazyflie bereitgestellte Topic *create\_log\_block* um einen LogBlock zu erstellen. Den Namen können sie frei wählen.

```
ros2 topic pub /cfID/create_log_block ....
```

Hier finden sie die auf der Crazyflie zur Verfügung stehenden Log-Variablen. In der Simulation sind davon nur ausgewählte verfügbar. In der Simulation können diese mittels einem Aufruf auf das Topic *cfID/get\_logging\_toc.info* abgefragt werden. Dieses Topic sollte bei einer echten nicht aufgerufen werden.

Auf beiden Plattformen vertreten sind die Variablen *stateEstimate.x*, *stateEstimate.y*, *stateEstimate.z*. Die Variable *range.zrange* ist auf einer echten Crazyflie nur dann verfügbar, wenn dort ein Z-ranger oder Flow -Deck angebracht ist.

Nachdem sie den LogBlock erstellt haben stellt ihnen die Middleware 3 neue Topics zur Verfügung (siehe Dokumentation). Der *name* ist dabei der zuvor von ihnen gewählte.

Starten sie den LogBlock über das *start* Topic und hören sie es anschließend ab:

```
ros2 topic echo cfID/log/name/data
```

## 6 Hardware Crazyflie

Um diesen Teil bearbeiten zu können benötigen sie eine Crazyflie sowie ein Crazyradio.

Eine Crazyflie wird eindeutig über ihre die Kombination aus *ID* und *Channel* identifiziert. Der Channel bildet dabei einen Teilbereich der Radiofrequenzen ab, auf welchem kommuniziert werden kann (siehe hier). Auf einem *Channel* darf daher immer nur eine Antenne gleichzeitig kommunizieren. Damit dies im Labor nicht zu Problemen führt sprechen sie sich mit ihren Kommilitonen ab, auf welchem *Channel* sie funken.

Der erste Schritt ist also auf ihrer Crazyflie den *Channel* einzustellen. Verwenden sie dazu den Crazyflie Client. Und folgen sie dem nächsten Teilkapitel.

### 6.1 Erste Verbindung

Schalten sie die Crazyflie an (Akku anstecken/per USB-Power verbinden/ Knopf an der Vorderseite) und stecken sie die Antenne in einen freien USB-Port an ihrem PC. Ihre Crazyflie ist auf der Unterseite mit einer ID beschriftet. Tragen sie diese in das Adress-Feld ein. Eine Crazyflie Adresse besteht dabei aus 5 Bytes. Die ersten vier sind dabei immer *E7*. In letzten Byte finden sie die ID (auch in Hex codiert). Für die ID = 1 lautet die Adresse also: *0xE7E7E7E701*. Drücken sie anschließend den Scan-Button, dieser überprüft durchsucht alle *Channel* nach der *Adresse*. Im Dropdown Menü sollte nun ihre Crazyflie angezeigt werden. Wählen sie diese aus und drücken sie Connect.

Konfigurieren sie nun den Channel: Drücken sie das Menüfeld *Connect*→*Configure* hier können sie nun den Channel einstellen. Der Channel solle als vielfaches von 20 gewählt werden in einem Bereich von 0 - 120 (e.g. 80, 100, 120).

Nach dem ändern des Channels müssen sie sich erneut mit der Crazyflie verbinden. Starten sie außerdem die Crazyflie neu.

Erkunden sie nun nochmal weiter den Crazyflie Client. Speziell im Tab *Console* können sie die Nachrichten welche die Crazyflie sendet sehen. Dazu muss der Konsolentab geöffnet sein, bevor sie sich verbinden. Außerdem können sie in diesem Tab einen sogenannten Propeller und Batterietest durchführen. Achten sie darauf, dass sich nichts im Bewegungsbereich der Propeller befindet, bevor sie diese durchführen.

### 6.2 Verbinden mit der Middleware

Nun sollen sie das erste mal mit der Dynamic Swarms Middleware eine Verbindung mit der Crazyflie aufbauen. **Sourcen** sie dazu in den jetzt verendeten Terminals das ds-crazyflies package, wie oben beschrieben.

In einem Terminal starten sie nun das Framework:

```
ros2 launch crazyflies framework.launch backend:=hardware
```

Sie sollten hier wenige Nachrichten bekommen die den Start-Prozess der Antenne und den weiteren Komponenten der Middleware beschreibt.

In einem weiteren Terminal rufen sie nun den Service */add\_crazyflie* des *crazyflie\_hardware\_gateways* auf.

```
ros2 service call /crazyflie_hardware_gateway/add_crazyflie ...
```

Dort müssen sie einige Parameter eintragen. Den *Channel* sowie die *ID* kennen sie bereits aus dem Client. Die *InitialPosition* können sie auf *[0.0,0.0,0.0]* belassen, da zunächst kein externes Tracking verwendet wird. Im Feld *type* tragen sie *default* ein. Eine zweite Option hier ist *tracked*, falls externes Tracking verwendet wird.

Im Framework Terminal sollten nun einige Nachrichten auftauchen. Wenn diese nicht auf einen fehlgeschlagenen Verbindungsversuch hinweisen ist die Verbindung geglückt. Eine der Crazyflie LEDs blinkt nun auch schnell in grün, was auf eine erfolgreiche Kommunikation hinweist.

Sie können ohne externes Tracking **nicht fliegen!** Sie können die Verbindung allerdings überprüfen indem sie Parameter auf der Crazyflie setzen.

## 6.3 Parameter

Wie in der Dokumentation beschrieben, sind die Parameter auf der Crazyflie direkt gekoppelt mit den ROS Parametern der Node mit dem Namen *cfID*. Über den folgenden Aufruf können also Parameter direkt auf der Crazyflie geändert werden:

```
ros2 parameter set \cfID parameterName value
```

Hier finden sie alle auf der Crazyflie verfügbaren Parameter. Solange an ihrer Crazyflie noch das LED-Deck angebracht ist eignet sich der Parameter *ring.effect*. Damit können sie den LED-Effekt verändern. Verwenden sie zunächst Effekt IDs von 1-3.

In der Simulation stehen keine Parameter zur Verfügung.

## 7 Appendix

Das Github-Repository zu *Dynamic Swarms* finden sie hier.

Die Dokumentation dazu hier.

Auch auf der offiziellen Website von Bitcraze finden sich einige nützliche Informationen.

Da die Navigation der Seite häufig mühsam ist lassen sich die meisten Dinge auch über eine Google-Suche finden.